

Gemini and CoPilot were both given the prompt below. Both GenAIs were accessed on a KU Linux machine using Firefox browser.

Prompt:

- “Generate a C program that will prioritize emails for a CEO using a max heap to implement a priority queue. The max heap will be array based and will be used to handle all email prioritizing. Create functions from scratch, do not use preexisting heap modules. The program will take a txt file containing commands as input and execute them in order based on their order in the file. These commands include: EMAIL, NEXT, READ, and COUNT. EMAIL is followed by a space and will contain a <sender category>, <subject line> and <date> in that order and separated by commas. The emails will be prioritized by their sender and date. The senders from highest to lowest priority are strings named Boss, Subordinate, Peer, ImportantPerson, OtherPerson. If there is more than one email from a sender, the newest email should take priority. <subject line> is a string which can contain spaces but not commas. <date> is in the format MM-DD-YYYY. NEXT displays the next email to be read in the format “Next Email:\n Sender:\n Subject:\n Date:\n”. READ will mark the highest priority email as read by removing it from the max heap and reordering the other emails accordingly. COUNT will display the current count of unread emails.”

Gemini's Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_LINE 256

// Structure to represent an Email
typedef struct {
    char sender[50];
    char subject[150];
    char date[11];
    int year;
    int month;
    int day;
    int sender_priority;
} Email;

// Structure to represent the Array-Based Max Heap
typedef struct {
    Email data[MAX_EMAILS];
```

```

    int size;
} MaxHeap;

// Function to determine sender priority (higher number = higher priority)

int get_sender_priority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "OtherPerson") == 0) return 1;
    return 0; // Default/Unknown
}

// Compare two emails. Returns > 0 if email 'a' has higher priority than 'b',
// < 0 if lower, and 0 if equal.
int compare_emails(Email a, Email b) {
    // 1. Compare sender priority first
    if (a.sender_priority != b.sender_priority) {
        return a.sender_priority - b.sender_priority;
    }

    // 2. If sender priority is the same, compare dates (Newest date = higher priority)
    if (a.year != b.year) {
        return a.year - b.year;
    }
    if (a.month != b.month) {
        return a.month - b.month;
    }
    return a.day - b.day;
}

// Swap two email elements
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

```

// Insert an email into the max heap
void max_heap_insert(MaxHeap *heap, Email e) {
    if (heap->size >= MAX_EMAILS) {
        printf("Error: Heap capacity exceeded.\n");
        return;
    }

    int i = heap->size++;
    heap->data[i] = e;

    // Bubble up
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (compare_emails(heap->data[i], heap->data[parent]) > 0) {
            //my comment: if the current email has greater priority than its parent, swap
positions
            swap(&heap->data[i], &heap->data[parent]);
            i = parent;
        } else {
            break; //my comment: exit when all is in order
        }
    }
}

// Heapify down helper for extraction
void max_heapify_down(MaxHeap *heap, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < heap->size && compare_emails(heap->data[left], heap->data[largest]) > 0) {
        largest = left;
    }
    if (right < heap->size && compare_emails(heap->data[right], heap->data[largest]) > 0)
    {
        largest = right;
    }

    if (largest != i) {

```

```

        swap(&heap->data[i], &heap->data[largest]);
        max_heapify_down(heap, largest);
    }
}

// Extract the highest priority email (READ command)
int max_heap_extract(MaxHeap *heap, Email *out) {
    if (heap->size <= 0) {
        return 0; // Empty heap
    }

    *out = heap->data[0];
    heap->data[0] = heap->data[--heap->size];
    max_heapify_down(heap, 0);
    return 1;
}

// Peek at the highest priority email without removing it (NEXT command)
int max_heap_peek(MaxHeap *heap, Email *out) {
    if (heap->size <= 0) {
        return 0;
    }

    *out = heap->data[0];
    return 1;
}

// Process command file inputs
void process_commands(const char *filename) {
    FILE *file = fopen(filename, "r");
    if (!file) {
        printf("Error: Could not open file %s\n", filename);
        return;
    }

    MaxHeap heap;
    heap.size = 0;

    char line[MAX_LINE];
    while (fgets(line, sizeof(line), file)) {

```

```

// Remove trailing newline/carriage return characters
line[strcspn(line, "\r\n")] = 0;

if (strncmp(line, "EMAIL", 5) == 0) {
    // Parse EMAIL command: EMAIL <sender category>, <subject line>, <MM-DD-YYYY>
    char *content = line + 6; // Skip "EMAIL "
    char *comma1 = strchr(content, ',');
    char *comma2 = strrchr(content, ',');

    if (comma1 && comma2 && comma1 != comma2) {
        Email new_email;

        // Extract Sender
        int len_sender = comma1 - content;
        strncpy(new_email.sender, content, len_sender);
        new_email.sender[len_sender] = '\0';

        // Extract Subject (trim surrounding spaces)
        char *subj_start = comma1 + 1;
        while (*subj_start == ' ') subj_start++;
        int len_subj = comma2 - subj_start;
        while (len_subj > 0 && subj_start[len_subj - 1] == ' ') len_subj--;
        strncpy(new_email.subject, subj_start, len_subj);
        new_email.subject[len_subj] = '\0';

        // Extract Date
        char *date_start = comma2 + 1;
        while (*date_start == ' ') date_start++;
        strcpy(new_email.date, date_start);

        // Parse Date numbers for comparison (MM-DD-YYYY)
        sscanf(new_email.date, "%d-%d-%d", &new_email.month, &new_email.day,
&new_email.year);

        // Set Priority weight based on sender
        new_email.sender_priority = get_sender_priority(new_email.sender);

        // Insert into Max Heap

```

```

        max_heap_insert(&heap, new_email);
    }
}
else if (strcmp(line, "NEXT") == 0) {
    Email next_email;
    if (max_heap_peek(&heap, &next_email)) {
        printf("Next Email:\n");
        printf("Sender: %s\n", next_email.sender);
        printf("Subject: %s\n", next_email.subject);
        printf("Date: %s\n", next_email.date);
    } else {
        printf("Next Email:\nNo unread emails.\n");
    }
}
else if (strcmp(line, "READ") == 0) {
    Email read_email;
    max_heap_extract(&heap, &read_email);
}
else if (strcmp(line, "COUNT") == 0) {
    printf("Unread Emails Count: %d\n", heap.size);
}
}

fclose(file);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <commands_file.txt>\n", argv[0]);
        return 1;
    }

    process_commands(argv[1]);

    return 0;
}

```

Copilot's Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_LINE 512

typedef struct {
    char sender[50];
    char subject[200];
    char date[11];    // MM-DD-YYYY
} Email;

typedef struct {
    Email heap[MAX_EMAILS];
    int size;
} MaxHeap;

/*-----
   Utility Functions
-----*/

int getSenderPriority(const char *sender) {
    if (strcmp(sender, "Boss") == 0)
        return 5;
    if (strcmp(sender, "Subordinate") == 0)
        return 4;
    if (strcmp(sender, "Peer") == 0)
        return 3;
    if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    if (strcmp(sender, "OtherPerson") == 0)
        return 1;

    return 0;
}

/*
```

Converts MM-DD-YYYY into YYYYMMDD integer
for easy date comparison.

*/

```
int convertDate(const char *date) {  
    int month, day, year;  
  
    sscanf(date, "%d-%d-%d", &month, &day, &year);  
  
    return year * 10000 + month * 100 + day;  
}
```

/*

Returns:
>0 if e1 has higher priority
<0 if e2 has higher priority
0 if equal

*/

```
int compareEmails(Email e1, Email e2) {  
  
    int p1 = getSenderPriority(e1.sender);  
    int p2 = getSenderPriority(e2.sender);  
  
    if (p1 != p2)  
        return p1 - p2;  
  
    int d1 = convertDate(e1.date);  
    int d2 = convertDate(e2.date);  
  
    return d1 - d2;  
}
```

```
void swap(Email *a, Email *b) {  
    Email temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

/*-----

Heap Functions

-----*/

```
void heapifyUp(MaxHeap *h, int index) {

    while (index > 0) {

        int parent = (index - 1) / 2;

        if (compareEmails(h->heap[index],
                        h->heap[parent]) > 0) {

            swap(&h->heap[index],
                &h->heap[parent]);

            index = parent;
        }
        else {
            break;
        }
    }
}
```

```
void heapifyDown(MaxHeap *h, int index) {

    while (1) {

        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < h->size &&
            compareEmails(h->heap[left],
                        h->heap[largest]) > 0) {

            largest = left;
        }

        if (right < h->size &&
            compareEmails(h->heap[right],
```

```

        h->heap[largest]) > 0) {

    largest = right;
}

if (largest != index) {

    swap(&h->heap[index],
        &h->heap[largest]);

    index = largest;
}
else {
    break;
}
}
}

```

```

void insertEmail(MaxHeap *h, Email email) {

```

```

    if (h->size >= MAX_EMAILS) {
        printf("Heap is full.\n");
        return;
    }

```

```

    h->heap[h->size] = email;
    heapifyUp(h, h->size);

```

```

    h->size++;
}

```

```

Email removeMax(MaxHeap *h) {

```

```

    Email emptyEmail = {"", "", ""};

```

```

    if (h->size == 0)
        return emptyEmail;

```

```

    Email maxEmail = h->heap[0];

```

```

h->size--;

h->heap[0] = h->heap[h->size];

heapifyDown(h, 0);

return maxEmail;
}

```

```

Email getMax(MaxHeap *h) {

```

```

    Email emptyEmail = {"", "", ""};

    if (h->size == 0)
        return emptyEmail;

    return h->heap[0];
}

```

```

/*-----
    Command Handlers
-----*/

```

```

void processEMAIL(MaxHeap *heap, char *data) {

```

```

    Email email;

    char *token;

    token = strtok(data, ",");
    if (token == NULL) return;
    strcpy(email.sender, token);

    token = strtok(NULL, ",");
    if (token == NULL) return;
    strcpy(email.subject, token);

    token = strtok(NULL, ",");

```

```
if (token == NULL) return;
```

```
while (*token == ' ')  
    token++;
```

```
strcpy(email.date, token);
```

```
insertEmail(heap, email);
```

```
}
```

```
void processNEXT(MaxHeap *heap) {
```

```
    if (heap->size == 0) {  
        printf("No unread emails.\n");  
        return;  
    }
```

```
    Email top = getMax(heap);
```

```
    printf("Next Email:\n");  
    printf("Sender: %s\n", top.sender);  
    printf("Subject: %s\n", top.subject);  
    printf("Date: %s\n", top.date);
```

```
}
```

```
void processREAD(MaxHeap *heap) {
```

```
    if (heap->size == 0) {  
        printf("No unread emails.\n");  
        return;  
    }
```

```
    removeMax(heap);
```

```
}
```

```
void processCOUNT(MaxHeap *heap) {
```

```
    printf("Unread Emails: %d\n", heap->size);
```

```
}
```

```

/*-----
Main
-----*/

int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s <inputfile>\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "r");

    if (fp == NULL) {
        printf("Unable to open file.\n");
        return 1;
    }

    MaxHeap heap;
    heap.size = 0;

    char line[MAX_LINE];

    while (fgets(line, sizeof(line), fp)) {

        line[strcspn(line, "\n")] = '\0';

        if (strncmp(line, "EMAIL ", 6) == 0) {

            char data[MAX_LINE];
            strcpy(data, line + 6);

            processEMAIL(&heap, data);
        }
        else if (strcmp(line, "NEXT") == 0) {

            processNEXT(&heap);
        }
        else if (strcmp(line, "READ") == 0) {

```

```

        processREAD(&heap);
    }
    else if (strcmp(line, "COUNT") == 0) {

        processCOUNT(&heap);
    }
}

fclose(fp);
return 0;
}

```

Analysis:

1. Gemini

- a. Correctness: Upon observation, the program seems like it is able to run properly. It separates aspects of the program into functions, and seems to account for edge cases like an empty MaxHeap. However, the program does not take any user input. Because of this, the name of the test file had to be written into the code directly. Despite this, the program was not able to compile.
- b. Execution Time $[O(n)]$: The execution time depends on the number of commands in the text txt file. There is no recursion. The main data types are two structs, one used to hold the Email information, and the other used to create the MaxHeap. The program is split into functions, which may increase or decrease its runtime.
- c. Space Complexity $[O(n)]$: There are about 200 lines of code. There are two defined structs, one for the Email, and the other for the MaxHeap. There are also many ints used for various purposes such as iteration and upheap/downheap.
- d. Maintainability: Comments exist, but they only summarize code blocks or very specific lines briefly; they could be more detailed. Comments describing common heap features mention their command equivalents (EMAIL, NEXT, READ). When, where and how pointers are used is not described at all, which could be confusing to some. Some variable names make their use evident (sender, filename), while others may be more vague. The name of the input file is hardcoded, which is a major issue as the user cannot use their own file. In the future, commands are used the the mainfile could be declared in other c files/header files to improve organization. Overall, the code seems a bit overcomplicated for the task in some parts.

2. CoPilot

- a. Correctness: Copilot's program also separates aspects of the program into functions. It does not ask for user input, and the file name needs to be enter into the code manually. The functions in the program can handle every command

when given in an expected and unexpected order. For example, when asked to display the next email with no emails in the inbox, the program outputs "No unread emails." When asked to count remaining emails before emails are added, the program outputs a count of 0. All emails given to the program are also properly sorted in order of priority.

- b. Execution Time $[O(n)]$: There is no recursion, and minimal to no use of for loops; most heap functions use while loops. There are little to no built-in methods used, all heap functions are written directly in the program.
- c. Space Complexity $[O(n)]$: The structs used in Copilot's code have smaller structs. There is no recursion, but various while loops are used in functions relating to the heap. Various ints are also used within functions for recursion and index calculation purposes. The only n dependent data type is the heap, but it has a size limit of 100; all of the large variables used in the program are size limited.
- d. Maintainability: Copilot's code has almost no comments except for those used to separate and name code blocks. There are no comments summarizing or explaining specific lines or blocks. In the `heapifyDown` function, a `while(1)` loop is used. This could be harmful if none of the cases in the loop are properly met, but this is accounted for with an `else{break}` inside the while loop. However, the code is overall more understandable than Gemini's.

Program Selected: Copilot

1. Justification

- a. Copilot's code is overall less complicated and easier to understand and analyze, which is an important trait to have when working on code that others will see. The functions are more legibly named and separated; the purpose of each function is clear. Structs are also smaller and only have what is necessary, and large data types are size limited.

2. Changes Made

- a. Comments added by me are indicated by "myC". Instead of using `argc` and `argv`, the program changed to take user input with `scanf`. This is less advanced but does what is needed. However, this does not seem to work as expected; an alternative is needed, so the file name must still be manually written in for now.